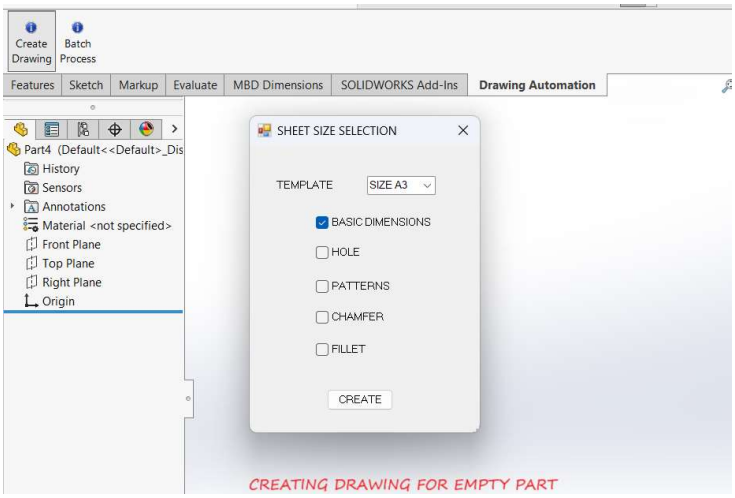
Error Handling

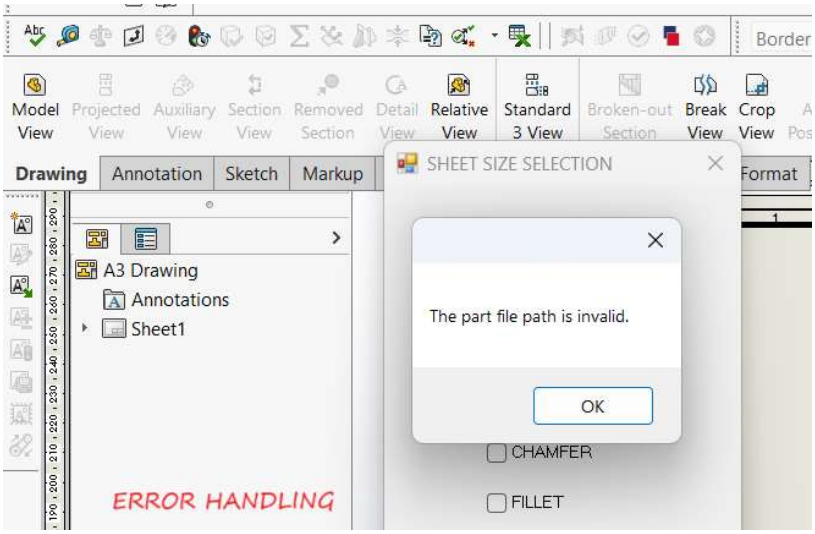
1. Add-In Registration Errors

- **Issue**: Failure to register the add-in in SolidWorks.
- **Handling**:
    - Exceptions during registration and unregistration are caught, and error messages are logged to the console.
    - Suggested Solutions:
        - Verify registry permissions.
        - Run SolidWorks as an administrator.

2. Document Validation

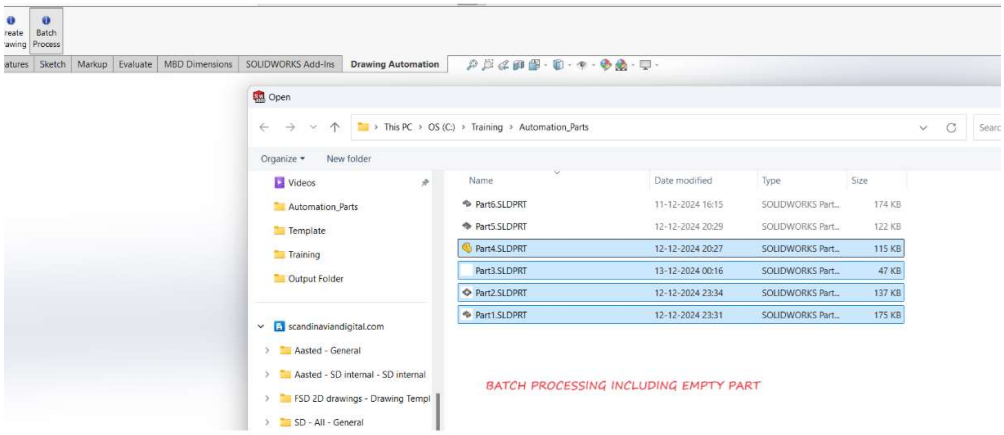
- **Issue**: An active document is invalid for drawing creation (e.g., there is no active document or an unsupported file type).
- **Handling**:
    - Before creating drawings, the add-in validates:
        - Whether an active document exists.
        - If the active document is a part or assembly.
    - If validation fails, a user-friendly message box is displayed with guidance to resolve the issue.





3. File Processing Errors

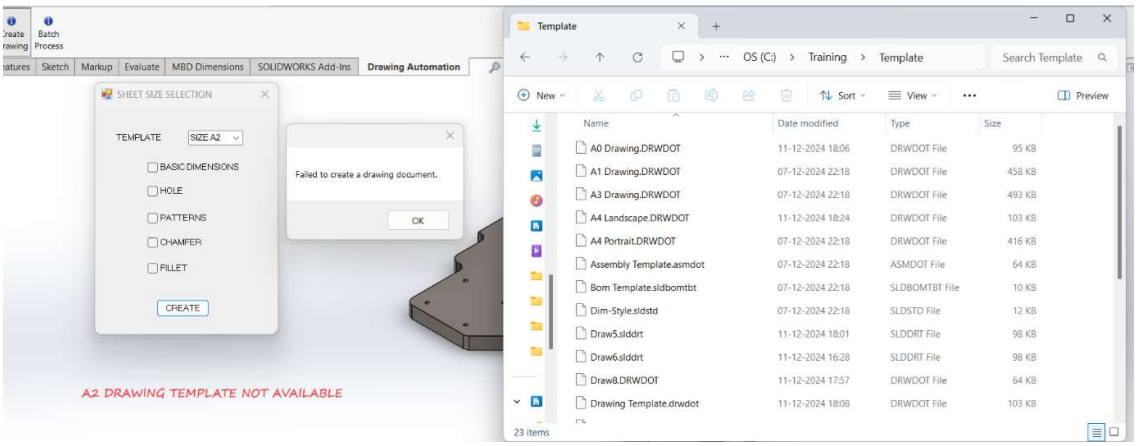
- **Issue**: Errors when processing part files during batch operations.
- **Handling**:
    - Invalid or unsupported file types are skipped, and an error message is displayed for the skipped file.
    - The batch process continues with the next valid file instead of terminating.





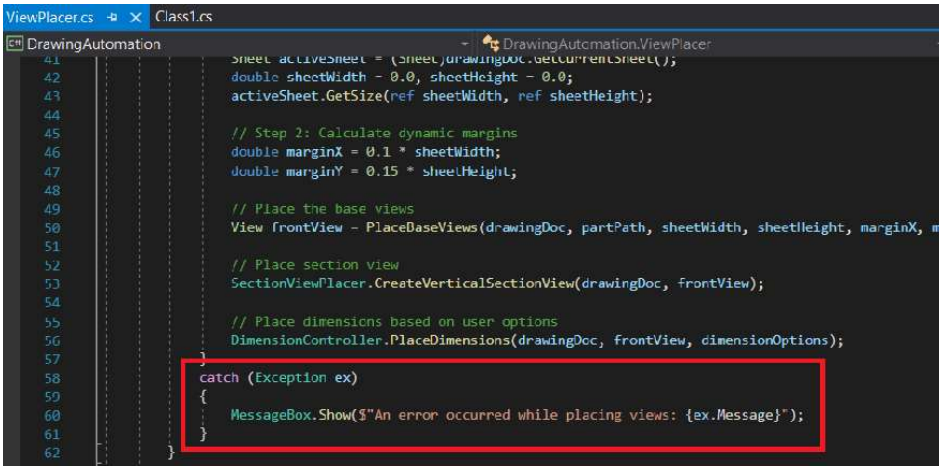
4. Template Loading Errors

- **Issue**: Failure to load the selected template.
- **Handling**:
    - If the template file is missing or inaccessible:
        - A message box notifies the user of the issue.
        - The process is stopped to prevent further errors.



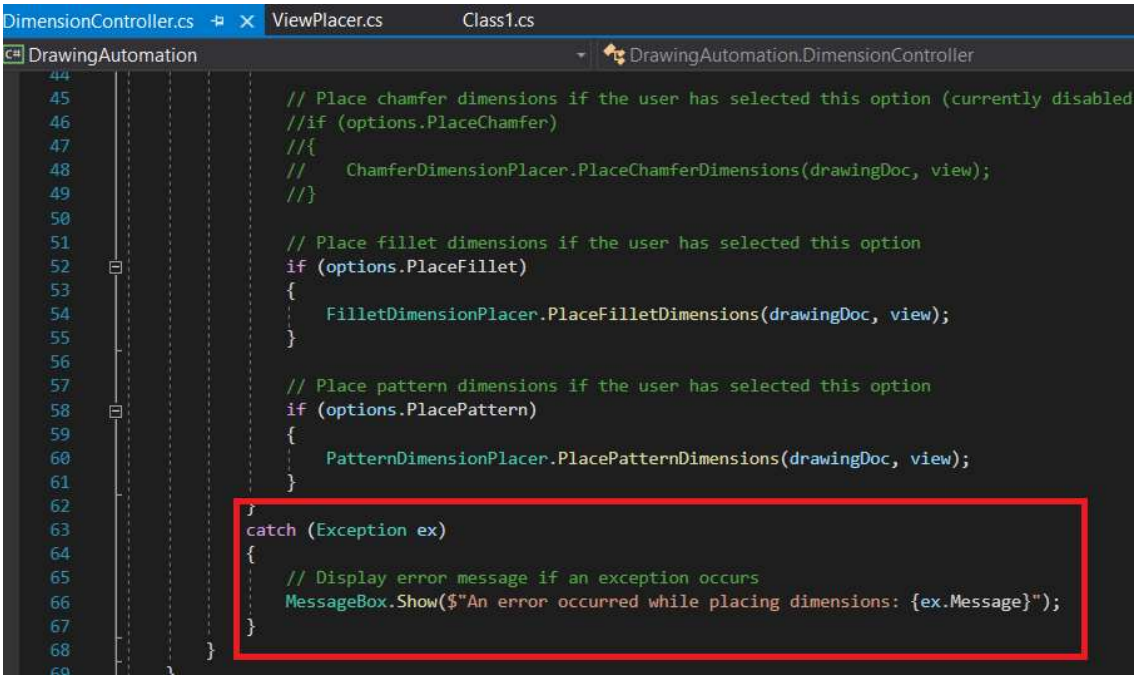
5. View Creation Errors

- **Issue**: Failure to place views (e.g., front, top, isometric).
- **Handling**:
    - Exceptions are caught if the view creation fails due to:
        - Invalid part geometry.
        - Unsupported part data.
    - The error is logged, and the user is notified with a detailed message.



6. Dimension Placement Errors

- **Issue**: Failure to place dimensions as intended.
- **Handling**:
    - If dimensions are not applied due to feature or view constraints:
        - The user is notified with specific details.
        - The process continues without interrupting other operations.



7. Batch Process Errors

- **Issue**: Errors encountered when processing multiple files in a batch.
- **Handling**:
    - Logs errors for individual files without halting the batch process.
    - Displays a summary of skipped files and reasons at the end of the operation.

```
public void BatchProcessCallback()
{
    try
    {
        using (OpenFileDialog dialog = new OpenFileDialog())
        {
            dialog.Multiselect = true;
            dialog.Filter = "SolidWorks Part Files (*.sldprt)|*.sldprt";

            if (dialog.ShowDialog() == DialogResult.OK)
            {
                string[] partFilePaths = dialog.FileNames;
                List<string> validPartFiles = partFilePaths
                    .Where(filePath => Path.GetExtension(filePath).ToLower() == ".sldprt")
                    .ToList();

                string templatePath = @"C:\Training\Template\A2 Drawing.DRWDOT";
                string saveLocation = @"C:\Training\Automation_Parts";

                foreach (string partFilePath in validPartFiles)
                {
                    PartDoc partDoc = (PartDoc)_swApp.OpenDoc6(partFilePath, (int)swDocumentTypes_e.swDocPART, 0, "", 0, 0);

                    if (partDoc != null)
                    {
                        DrawingPreparationBatch.PrepareDrawing(_swApp, templatePath, saveLocation);
                        _swApp.CloseDoc(Path.GetFileName(partFilePath));
                        _swApp.CloseAllDocuments(true);
                    }
                    else
                    {
                        MessageBox.Show($"Failed to open part file: {partFilePath}");
                    }
                }
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Error during batch processing: {ex.Message}", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

8. File Save Errors

- **Issue**: Failure to save generated drawings.
- **Handling**:
    - Verifies permissions in the save location before attempting to save.
    - Displays a clear error message if saving fails.

```
DrawingPreparationBatch.cs  Class1.cs
DrawingAutomation  DrawingAutomation.DrawingPreparationBatch  PrepareDrawing(ISld
                // Step 4: Generate the file path for saving the drawing
                string drawingFileName = Path.Combine(saveLocation, $"{Path.GetFileNameWithoutExtension(activeDoc.GetPathName())}.slddrw");

                // Step 5: Save the drawing document
                bool saveResult = drawingDoc.SaveAs(drawingFileName);
                if (!saveResult)
                {
                    MessageBox.Show($"Failed to save the drawing: {drawingFileName}");
                }

                // Step 6: Close the drawing document
                swApp.CloseDoc(drawingDoc.GetTitle());
            }
            catch (Exception ex)
            {
                // Log and display any errors encountered during the process
                MessageBox.Show($"An error occurred while preparing the drawing: {ex.Message}", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }
    }
}
```